NLP Unit 1.0 - Text Preprocessing

This notebook covers all the essential text preprocessing techniques used in Natural Language Processing.

Topics Covered:

1. **Tokenization** - Breaking text into words/sentences
2. **Stop Words Removal** - Removing common words
3. **Stemming** - Reducing words to root form (crude)
4. **Lemmatization** - Reducing words to dictionary base form (accurate)
5. **Text Cleaning** - Removing punctuation, special characters, etc.
6. **Normalization** - Lowercasing, handling numbers

Installation & Setup

```
# Install required libraries
!pip install nltk spacy
```

```
Requirement already satisfied: nltk in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (3.9.2)
Requirement already satisfied: spacy in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (3.8.11)
Requirement already satisfied: click in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from nltk) (2026.1.15)
Requirement already satisfied: tqdm in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from nltk) (4.67.3)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in
```

```
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (0.24.0)
Requirement already satisfied: numpy>=1.19.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (2.4.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (2.32.5)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (2.12.5)
Requirement already satisfied: jinja2 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (82.0.0)
Requirement already satisfied: packaging>=20.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from spacy) (26.0)
Requirement already satisfied: annotated-types>=0.6.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from pydantic!=1.8,!
=1.8.1,<3.0.0,>=1.7.4->spacy) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from pydantic!=1.8,!
=1.8.1,<3.0.0,>=1.7.4->spacy) (2.41.5)
```

```
Requirement already satisfied: typing-extensions>=4.14.1 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from pydantic!=1.8,!
=1.8.1,<3.0.0,>=1.7.4->spacy) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from pydantic!=1.8,!
=1.8.1,<3.0.0,>=1.7.4->spacy) (0.4.2)
Requirement already satisfied: charset_normalizer<4,>=2 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy)
(3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy)
(3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy)
(2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy)
(2026.1.4)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from thinc<8.4.0,>=8.3.4->spacy)
(1.3.3)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from thinc<8.4.0,>=8.3.4->spacy)
(0.1.5)
Requirement already satisfied: typer>=0.24.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from typer-slim<1.0.0,>=0.3.0->spacy)
(0.24.0)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from weasel<0.5.0,>=0.4.2->spacy)
(0.23.0)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from weasel<0.5.0,>=0.4.2->spacy)
(7.5.0)
Requirement already satisfied: wrapt in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from smart-open<8.0.0,>=5.2.1-
>weasel<0.5.0,>=0.4.2->spacy) (2.1.1)
Requirement already satisfied: shellingham>=1.3.0 in
```

```
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from typer>=0.24.0->typer-
slim<1.0.0,>=0.3.0->spacy) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from typer>=0.24.0->typer-
slim<1.0.0,>=0.3.0->spacy) (14.3.2)
Requirement already satisfied: annotated-doc>=0.0.2 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from typer>=0.24.0->typer-
slim<1.0.0,>=0.3.0->spacy) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from rich>=12.3.0->typer>=0.24.0-
>typer-slim<1.0.0,>=0.3.0->spacy) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from rich>=12.3.0->typer>=0.24.0-
>typer-slim<1.0.0,>=0.3.0->spacy) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from markdown-it-py>=2.2.0-
>rich>=12.3.0->typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (0.1.2)
Requirement already satisfied: MarkupSafe>=2.0 in
/Users/damirdarasu/Library/CloudStorage/OneDrive-Aziro/mynotebook/.ven
v/lib/python3.13/site-packages (from jinja2->spacy) (3.0.3)
```

```python
# Download spaCy English model
!python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Using cached
https://github.com/explosion/spacy-models/releases/download/en_core_we
b_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
```

```python
# Import libraries
import nltk
import spacy
import re
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, LancasterStemmer
from nltk.stem import WordNetLemmatizer

# Download NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
nltk.download('omw-1.4')

# Load spaCy model
nlp = spacy.load("en_core_web_sm")

print("✓ All libraries loaded successfully!")
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     /Users/damirdarasu/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     /Users/damirdarasu/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     /Users/damirdarasu/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     /Users/damirdarasu/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!

✓ All libraries loaded successfully!
```

---

1. Tokenization

Tokenization is the process of breaking down text into smaller units (tokens) like words or sentences.

Types of Tokenization:

- **Word Tokenization**: Splits text into individual words
- **Sentence Tokenization**: Splits text into sentences
- **N-gram Tokenization**: Creates sequences of n words

```
# Sample text
text = "Hello! This is a simple text preprocessing example. Let's see
how tokenization works. It's really useful for NLP!"

print("Original Text:")
print(text)
print("\n" + "="*60 + "\n")

# Word Tokenization
word_tokens = word_tokenize(text)
print("Word Tokens:")
print(word_tokens)
print(f"\nTotal words: {len(word_tokens)}")

print("\n" + "="*60 + "\n")
```

```python
# Sentence Tokenization
sent_tokens = sent_tokenize(text)
print("Sentence Tokens:")
for i, sent in enumerate(sent_tokens, 1):
    print(f"{i}. {sent}")
```

```
Original Text:
Hello! This is a simple text preprocessing example. Let's see how
tokenization works. It's really useful for NLP!

============================================================

Word Tokens:
['Hello', '!', 'This', 'is', 'a', 'simple', 'text', 'preprocessing',
'example', '.', 'Let', "'s", 'see', 'how', 'tokenization', 'works',
'.', 'It', "'s", 'really', 'useful', 'for', 'NLP', '!']

Total words: 24

============================================================

Sentence Tokens:
1. Hello!
2. This is a simple text preprocessing example.
3. Let's see how tokenization works.
4. It's really useful for NLP!
```

---

2. Stop Words Removal

Stop Words are common words that carry little semantic meaning (e.g., "the", "is", "a", "an").

⚠ **Important Note**: Not always recommended to remove stop words! In some cases like sentiment analysis, words like "not", "no", "never" are crucial for understanding meaning.

```python
# Get English stop words
stop_words = set(stopwords.words('english'))

print(f"Total stop words in NLTK: {len(stop_words)}")
print("\nFirst 20 stop words:")
print(list(stop_words)[:20])
```

```
Total stop words in NLTK: 198

First 20 stop words:
["they've", 'they', 've', 'off', 'hasn', "i'll", "hadn't", 'll',
'same', 'are', 'from', 'being', 'so', "they'd", 'shan', 'under',
'too', 'my', 'if', 'can']
```

```
# Example: Remove stop words
text = "I love Natural Language Processing"
tokens = word_tokenize(text.lower())

print("Original tokens:", tokens)

# Remove stop words
filtered_tokens = [word for word in tokens if word not in stop_words]

print("After removing stop words:", filtered_tokens)
print(f"\n✓ Note: 'I' is a stop word and was removed")
print(f"✓ Content words (love, natural, language, processing) are
kept!")
```

```
Original tokens: ['i', 'love', 'natural', 'language', 'processing']
After removing stop words: ['love', 'natural', 'language',
'processing']

✓ Note: 'I' is a stop word and was removed
✓ Content words (love, natural, language, processing) are kept!
```

---

3. Stemming

Stemming uses crude heuristic rules to chop off word endings.

Characteristics:

- ⚡ Fast (rule-based)
- [] May produce non-words (e.g., "studies" → "studi")
- [] Context-independent
- [] Good for: Search engines, approximate matching

Popular Stemmers:

- **Porter Stemmer**: Most commonly used, less aggressive
- **Lancaster Stemmer**: More aggressive, can produce shorter stems

```
# Initialize stemmers
porter = PorterStemmer()
lancaster = LancasterStemmer()

# Test words
words = ["running", "runs", "ran", "runner", "easily", "fairly",
"studies", "studying", "caring", "better"]

print("Word".ljust(15), "Porter".ljust(15), "Lancaster")
print("="*50)

for word in words:
```

```
    porter_stem = porter.stem(word)
    lancaster_stem = lancaster.stem(word)
    print(f"{word.ljust(15)} {porter_stem.ljust(15)}
{lancaster_stem}")
```

```
Word            Porter          Lancaster
==================================================
running         run             run
runs            run             run
ran             ran             ran
runner          runner          run
easily          easili          easy
fairly          fairli          fair
studies         studi           study
studying        studi           study
caring          care            car
better          better          bet
```

```
# Stemming example with sentence
def stem_text(text):
    porter = PorterStemmer()

    # Tokenize and stem
    tokens = word_tokenize(text.lower())
    stemmed = [porter.stem(word) for word in tokens]

    print("Original:", tokens)
    print("Stemmed: ", stemmed)

    return ' '.join(stemmed)

text = "The runners are running in the park. They ran better than
yesterday."
result = stem_text(text)
print("\nResult:", result)
```

```
Original: ['the', 'runners', 'are', 'running', 'in', 'the', 'park',
'.', 'they', 'ran', 'better', 'than', 'yesterday', '.']
Stemmed:  ['the', 'runner', 'are', 'run', 'in', 'the', 'park', '.',
'they', 'ran', 'better', 'than', 'yesterday', '.']

Result: the runner are run in the park . they ran better than
yesterday .
```

---

4. Lemmatization

Lemmatization uses vocabulary and morphological analysis to return the dictionary base form.

Characteristics:

- [] Accurate (linguistic approach)
- [] Always produces valid dictionary words
- [] Slower (requires dictionary lookup)
- [] Context-aware (considers part of speech)
- [] Good for: Text analytics, NLP models requiring valid words

NLTK vs spaCy Lemmatization:

- **NLTK**: Uses WordNet lemmatizer
- **spaCy**: More context-aware, considers POS tags automatically

```python
# NLTK Lemmatization
lemmatizer = WordNetLemmatizer()

words = ["running", "runs", "ran", "runner", "easily", "fairly",
"studies", "studying", "caring", "better", "am", "is", "are"]

print("Word".ljust(15), "Lemma (noun)".ljust(15), "Lemma (verb)")
print("="*50)

for word in words:
    lemma_noun = lemmatizer.lemmatize(word, pos='n')  # noun
    lemma_verb = lemmatizer.lemmatize(word, pos='v')  # verb
    print(f"{word.ljust(15)} {lemma_noun.ljust(15)} {lemma_verb}")

print("\n⚠ Note: NLTK lemmatizer requires POS tag to work properly!")
print("Without POS tag, 'better' stays 'better' instead of becoming
'good'")
```

```
Word            Lemma (noun)    Lemma (verb)
==================================================
running         running         run
runs            run             run
ran             ran             run
runner          runner          runner
easily          easily          easily
fairly          fairly          fairly
studies         study           study
studying        studying        study
caring          caring          care
better          better          better
am              am              be
is              is              be
are             are             be

⚠ Note: NLTK lemmatizer requires POS tag to work properly!
Without POS tag, 'better' stays 'better' instead of becoming 'good'
```

```python
# spaCy Lemmatization (More powerful - automatically detects POS)
def lemmatize_with_spacy(text):
    doc = nlp(text)

    print("Token".ljust(15), "Lemma".ljust(15), "POS")
    print("="*50)

    for token in doc:
        print(f"{token.text.ljust(15)} {token.lemma_.ljust(15)}
{token.pos_}")

    lemmatized = " ".join([token.lemma_ for token in doc])
    return lemmatized

text = "My name is Damir and I am running in the park. I have better
running shoes than you."
print(f"Original: {text}\n")

result = lemmatize_with_spacy(text)
print(f"\nLemmatized Result: {result}")
```

```
Original: My name is Damir and I am running in the park. I have better
running shoes than you.

Token           Lemma           POS
==================================================
My              my              PRON
name            name            NOUN
is              be              AUX
Damir           Damir           PROPN
and             and             CCONJ
I               I               PRON
am              be              AUX
running         run             VERB
in              in              ADP
the             the             DET
park            park            NOUN
.               .               PUNCT
I               I               PRON
have            have            VERB
better          well            ADV
running         run             VERB
shoes           shoe            NOUN
than            than            ADP
you             you             PRON
.               .               PUNCT

Lemmatized Result: my name be Damir and I be run in the park . I have
well run shoe than you .
```

5. Stemming vs Lemmatization Comparison

Let's compare both techniques side by side:

```
def compare_stem_vs_lemma(text):
    # Stemming
    porter = PorterStemmer()
    tokens = word_tokenize(text.lower())
    stemmed = [porter.stem(word) for word in tokens]

    # Lemmatization
    doc = nlp(text)
    lemmatized = [token.lemma_ for token in doc]

    print("Original Text:", text)
    print("\n" + "="*60)
    print("\nOriginal Tokens:", tokens)
    print("\nStemmed (Porter):", stemmed)
    print("\nLemmatized (spaCy):", lemmatized)

    # Find differences
    print("\n" + "="*60)
    print("\nDifferences:")
    for orig, stem, lemma in zip(tokens, stemmed, lemmatized):
        if stem != lemma:
            print(f"  '{orig}' → Stem: '{stem}' | Lemma: '{lemma}'")

compare_stem_vs_lemma("The studies show that running improves health.
Better results were seen in caring individuals.")
```

```
Original Text: The studies show that running improves health. Better
results were seen in caring individuals.

============================================================

Original Tokens: ['the', 'studies', 'show', 'that', 'running',
'improves', 'health', '.', 'better', 'results', 'were', 'seen', 'in',
'caring', 'individuals', '.']

Stemmed (Porter): ['the', 'studi', 'show', 'that', 'run', 'improv',
'health', '.', 'better', 'result', 'were', 'seen', 'in', 'care',
'individu', '.']

Lemmatized (spaCy): ['the', 'study', 'show', 'that', 'running',
'improve', 'health', '.', 'well', 'result', 'be', 'see', 'in',
'caring', 'individual', '.']

============================================================
```

```
Differences:
  'studies' → Stem: 'studi' | Lemma: 'study'
  'running' → Stem: 'run' | Lemma: 'running'
  'improves' → Stem: 'improv' | Lemma: 'improve'
  'better' → Stem: 'better' | Lemma: 'well'
  'were' → Stem: 'were' | Lemma: 'be'
  'seen' → Stem: 'seen' | Lemma: 'see'
  'caring' → Stem: 'care' | Lemma: 'caring'
  'individuals' → Stem: 'individu' | Lemma: 'individual'
```

---

6. Complete Text Preprocessing Pipeline

Combining all techniques into a comprehensive preprocessing function:

```python
def preprocess_text(text, remove_stopwords=True,
use_lemmatization=True):
    """
    Complete text preprocessing pipeline

    Steps:
    1. Lowercase
    2. Remove punctuation and special characters
    3. Tokenize
    4. Remove stop words (optional)
    5. Stemming or Lemmatization
    """
    print("="*60)
    print("TEXT PREPROCESSING PIPELINE")
    print("="*60)

    # Step 1: Lowercase
    text = text.lower()
    print("\n1. After Lowercasing:")
    print(f"   {text}")

    # Step 2: Remove punctuation
    text_clean = re.sub(r'[^\w\s]', '', text)
    print("\n2. After Removing Punctuation:")
    print(f"   {text_clean}")

    # Step 3: Tokenize
    tokens = word_tokenize(text_clean)
    print("\n3. After Tokenization:")
    print(f"   {tokens}")

    # Step 4: Remove stop words
```

```python
    if remove_stopwords:
        stop_words = set(stopwords.words('english'))
        tokens = [word for word in tokens if word not in stop_words]
        print("\n4. After Stop Words Removal:")
        print(f"   {tokens}")

    # Step 5: Lemmatization or Stemming
    if use_lemmatization:
        doc = nlp(' '.join(tokens))
        tokens = [token.lemma_ for token in doc]
        print("\n5. After Lemmatization:")
        print(f"   {tokens}")
    else:
        porter = PorterStemmer()
        tokens = [porter.stem(word) for word in tokens]
        print("\n5. After Stemming:")
        print(f"   {tokens}")

    # Final result
    result = ' '.join(tokens)
    print("\n" + "="*60)
    print("FINAL RESULT:")
    print(f"   {result}")
    print("="*60)

    return result

# Test the pipeline
text = "Hello! This is a simple text preprocessing example. Let's see
how it works with multiple sentences."
result = preprocess_text(text, remove_stopwords=True,
use_lemmatization=True)
```

```
============================================================
TEXT PREPROCESSING PIPELINE
============================================================

1. After Lowercasing:
   hello! this is a simple text preprocessing example. let's see how
it works with multiple sentences.

2. After Removing Punctuation:
   hello this is a simple text preprocessing example lets see how it
works with multiple sentences

3. After Tokenization:
   ['hello', 'this', 'is', 'a', 'simple', 'text', 'preprocessing',
'example', 'lets', 'see', 'how', 'it', 'works', 'with', 'multiple',
'sentences']
```

```
4. After Stop Words Removal:
   ['hello', 'simple', 'text', 'preprocessing', 'example', 'lets',
'see', 'works', 'multiple', 'sentences']

5. After Lemmatization:
   ['hello', 'simple', 'text', 'preprocesse', 'example', 'let', 'see',
'work', 'multiple', 'sentence']

============================================================
FINAL RESULT:
   hello simple text preprocesse example let see work multiple
sentence
============================================================
```

---

7. Additional Preprocessing Techniques

```python
# Other common preprocessing techniques

def advanced_preprocessing(text):
    print("Original Text:")
    print(text)
    print("\n" + "="*60 + "\n")

    # Remove URLs
    text = re.sub(r'http\S+|www.\S+', '', text)
    print("After removing URLs:")
    print(text)
    print()

    # Remove numbers
    text_no_nums = re.sub(r'\d+', '', text)
    print("After removing numbers:")
    print(text_no_nums)
    print()

    # Remove extra whitespace
    text_clean = re.sub(r'\s+', ' ', text_no_nums).strip()
    print("After removing extra spaces:")
    print(text_clean)
    print()

    # Remove special characters (keep only alphanumeric and spaces)
    text_final = re.sub(r'[^a-zA-Z\s]', '', text_clean)
    print("After removing special characters:")
    print(text_final)

    return text_final
```

```
sample = "Check this out: https://www.example.com! I scored 95% on my
test. Amazing!!! □"
result = advanced_preprocessing(sample)
```

```
Original Text:
Check this out: https://www.example.com! I scored 95% on my test.
Amazing!!! □

============================================================

After removing URLs:
Check this out:  I scored 95% on my test. Amazing!!! □

After removing numbers:
Check this out:  I scored % on my test. Amazing!!! □

After removing extra spaces:
Check this out: I scored % on my test. Amazing!!! □

After removing special characters:
Check this out I scored  on my test Amazing
```

---

8. Practice Exercise

Try preprocessing the following text with different configurations:

```
# Practice text
practice_text = """
Natural Language Processing (NLP) is a fascinating field!
It's helping computers understand human languages better.
Studies show that NLP applications are growing rapidly.
Visit https://nlp.stanford.edu for more info.
Isn't it amazing? □
"""

print("PRACTICE TEXT:")
print(practice_text)
print("\n" + "="*60 + "\n")

# Try different configurations
print("Configuration 1: With stop words removal + Lemmatization")
result1 = preprocess_text(practice_text, remove_stopwords=True,
use_lemmatization=True)

print("\n\n")

print("Configuration 2: Without stop words removal + Stemming")
```

```
result2 = preprocess_text(practice_text, remove_stopwords=False,
use_lemmatization=False)
```

```
PRACTICE TEXT:

Natural Language Processing (NLP) is a fascinating field!
It's helping computers understand human languages better.
Studies show that NLP applications are growing rapidly.
Visit https://nlp.stanford.edu for more info.
Isn't it amazing? 


============================================================

Configuration 1: With stop words removal + Lemmatization
============================================================
TEXT PREPROCESSING PIPELINE
============================================================

1. After Lowercasing:

natural language processing (nlp) is a fascinating field!
it's helping computers understand human languages better.
studies show that nlp applications are growing rapidly.
visit https://nlp.stanford.edu for more info.
isn't it amazing? 


2. After Removing Punctuation:

natural language processing nlp is a fascinating field
its helping computers understand human languages better
studies show that nlp applications are growing rapidly
visit httpsnlpstanfordedu for more info
isnt it amazing


3. After Tokenization:
   ['natural', 'language', 'processing', 'nlp', 'is', 'a',
'fascinating', 'field', 'its', 'helping', 'computers', 'understand',
'human', 'languages', 'better', 'studies', 'show', 'that', 'nlp',
'applications', 'are', 'growing', 'rapidly', 'visit',
'httpsnlpstanfordedu', 'for', 'more', 'info', 'isnt', 'it', 'amazing']

4. After Stop Words Removal:
   ['natural', 'language', 'processing', 'nlp', 'fascinating',
'field', 'helping', 'computers', 'understand', 'human', 'languages',
'better', 'studies', 'show', 'nlp', 'applications', 'growing',
'rapidly', 'visit', 'httpsnlpstanfordedu', 'info', 'isnt', 'amazing']

5. After Lemmatization:
```

```
   ['natural', 'language', 'processing', 'nlp', 'fascinating',
'field', 'help', 'computer', 'understand', 'human', 'language',
'well', 'study', 'show', 'nlp', 'application', 'grow', 'rapidly',
'visit', 'httpsnlpstanfordedu', 'info', 'be', 'not', 'amazing']

================================================================
FINAL RESULT:
   natural language processing nlp fascinating field help computer
understand human language well study show nlp application grow rapidly
visit httpsnlpstanfordedu info be not amazing
================================================================



Configuration 2: Without stop words removal + Stemming
================================================================
TEXT PREPROCESSING PIPELINE
================================================================

1. After Lowercasing:

natural language processing (nlp) is a fascinating field!
it's helping computers understand human languages better.
studies show that nlp applications are growing rapidly.
visit https://nlp.stanford.edu for more info.
isn't it amazing? 


2. After Removing Punctuation:

natural language processing nlp is a fascinating field
its helping computers understand human languages better
studies show that nlp applications are growing rapidly
visit httpsnlpstanfordedu for more info
isnt it amazing


3. After Tokenization:
   ['natural', 'language', 'processing', 'nlp', 'is', 'a',
'fascinating', 'field', 'its', 'helping', 'computers', 'understand',
'human', 'languages', 'better', 'studies', 'show', 'that', 'nlp',
'applications', 'are', 'growing', 'rapidly', 'visit',
'httpsnlpstanfordedu', 'for', 'more', 'info', 'isnt', 'it', 'amazing']

5. After Stemming:
   ['natur', 'languag', 'process', 'nlp', 'is', 'a', 'fascin',
'field', 'it', 'help', 'comput', 'understand', 'human', 'languag',
'better', 'studi', 'show', 'that', 'nlp', 'applic', 'are', 'grow',
'rapidli', 'visit', 'httpsnlpstanfordedu', 'for', 'more', 'info',
'isnt', 'it', 'amaz']
```

```
============================================================
FINAL RESULT:
   natur languag process nlp is a fascin field it help comput
understand human languag better studi show that nlp applic are grow
rapidli visit httpsnlpstanfordedu for more info isnt it amaz
============================================================
```

Summary

Key Takeaways:

1. **Tokenization**: Breaking text into words/sentences - essential first step
2. **Stop Words**: Common words with little meaning - remove with caution!
3. **Stemming**: Fast but crude, may create non-words
4. **Lemmatization**: Accurate but slower, always creates valid words
5. **Choose wisely**:
   - Use **Stemming** for: Speed-critical apps, search engines
   - Use **Lemmatization** for: Accuracy-critical apps, text analytics

When to Apply Each Technique:

Technique	Use Case
Tokenization	Always needed as first step
Stop Words Removal	Text classification, topic modeling (NOT for sentiment analysis)
Stemming	Search engines, information retrieval
Lemmatization	Text analytics, machine translation, QA systems
Lowercasing	Almost always (except for NER, POS tagging)
Punctuation Removal	Depends on task (keep for sentiment)

Remember:

> **"There's no one-size-fits-all preprocessing pipeline. Choose techniques based on your specific NLP task and data characteristics!"**